

Educational Cryptanalysis of the Merkle–Hellman Knapsack Cryptosystem Using QUBO and Annealing Methods

Jakub ZwoniarSKI

May 10, 2026

Abstract

This report presents an educational implementation of a subset-sum attack model for the Merkle–Hellman knapsack cryptosystem. A public knapsack ciphertext is treated as a subset-sum instance, encoded as a Quadratic Unconstrained Binary Optimization problem, and solved with exact and heuristic methods. The compared approaches are exhaustive brute force search, an exact QUBO solver used for validation, classical simulated annealing, and a local simulated quantum annealing style solver. The experiments are intentionally limited to small instances. Their purpose is to validate the mathematical model, compare solver behavior, and illustrate how a cryptographic problem can be expressed as binary optimization rather than to perform practical cryptanalysis of modern cryptographic systems.

Contents

1	Introduction	3
2	Merkle–Hellman Knapsack Cryptosystem	3
3	Subset-Sum as a QUBO Problem	4
4	Solvers	5
4.1	Brute Force	5
4.2	Exact QUBO Solver	5
4.3	Simulated Annealing	5
4.4	Local Simulated Quantum Annealing Style Solver	5
5	Implementation and Validation	5
6	Experimental Setup	6
7	Results	7
7.1	Exact-Hit Rate	7
7.2	Runtime	8
7.3	Objective Gap	9
7.4	Hamming Distance	10

8 Numerical Summary	10
9 Discussion	11
10 Limitations	11
11 Conclusion	12

1 Introduction

The subset-sum problem is a classical combinatorial problem. Given positive integer weights

$$a_1, a_2, \dots, a_n$$

and a target value C , the task is to find binary variables

$$x_i \in \{0, 1\}$$

such that

$$\sum_{i=1}^n a_i x_i = C.$$

The problem is simple to state but difficult in the general case. It is also closely related to knapsack-based cryptography.

The Merkle–Hellman cryptosystem is a historically important trapdoor knapsack construction. Legal decryption is easy with the private key, because the private key contains a superincreasing sequence. Without the trapdoor, the public problem resembles a general subset-sum instance. This makes the construction useful as an educational example of how cryptographic structure can be hidden inside a combinatorial problem.

The goal of this project is to build a reproducible demonstration of this relationship. The public Merkle–Hellman ciphertext is converted into a subset-sum instance, the subset-sum objective is written as a QUBO model, and several solvers are compared on small benchmark instances.

The main contributions are:

- implementation of a toy Merkle–Hellman cryptosystem,
- construction of subset-sum attack instances from public ciphertexts,
- QUBO formulation of the subset-sum objective,
- comparison of brute force, simulated annealing, and simulated quantum annealing style methods,
- validation of benchmark outputs and generation of report-ready plots.

2 Merkle–Hellman Knapsack Cryptosystem

The private key in the Merkle–Hellman cryptosystem is a superincreasing sequence

$$w_1, w_2, \dots, w_n,$$

where every element is greater than the sum of all previous elements:

$$w_i > \sum_{j=1}^{i-1} w_j.$$

This property makes subset-sum easy to solve greedily when the private sequence is known.

A modulus m and a multiplier r are chosen such that

$$m > \sum_{i=1}^n w_i$$

and

$$\gcd(r, m) = 1.$$

The public key is computed as

$$a_i = rw_i \bmod m.$$

For a binary message

$$x = (x_1, x_2, \dots, x_n),$$

encryption is the public knapsack sum

$$C = \sum_{i=1}^n a_i x_i.$$

Legal decryption uses the modular inverse $r^{-1} \bmod m$. Multiplying the ciphertext by this inverse gives

$$C' = Cr^{-1} \bmod m,$$

which corresponds to a subset-sum instance over the private superincreasing sequence. The original bits can then be recovered greedily.

In the benchmark, the attack solvers do not use the trapdoor. They only receive the public weights a_i and target C . Therefore, the recovery problem is treated as a general subset-sum problem.

3 Subset-Sum as a QUBO Problem

For public weights a_i , binary variables x_i , and target ciphertext C , the objective is

$$E(x) = \left(\sum_{i=1}^n a_i x_i - C \right)^2.$$

An exact subset-sum solution satisfies

$$E(x) = 0.$$

Expanding the square gives

$$E(x) = C^2 + \sum_{i=1}^n (a_i^2 - 2Ca_i)x_i + \sum_{i < j} 2a_i a_j x_i x_j.$$

This is a Quadratic Unconstrained Binary Optimization model. The constant term C^2 does not change the minimizing bit vector, but it is kept in the Binary Quadratic Model so that the energy remains equal to the original squared residual.

After sampling, the implementation recomputes the final objective using integer arithmetic:

$$\left(\sum_i a_i x_i - C \right)^2.$$

This avoids classifying a correct solution incorrectly due to floating point roundoff in BQM energies.

4 Solvers

4.1 Brute Force

The brute force solver evaluates all 2^n possible bit vectors. It is exact and provides the correctness baseline for the small benchmark instances. Its cost is exponential, so it is not scalable, but for $n \leq 16$ it is practical enough for validation.

4.2 Exact QUBO Solver

An exact Binary Quadratic Model solver is used to validate tiny QUBO models. It is not treated as a scalable attack method. Its role is to confirm that the QUBO model has the same minimizers as the original subset-sum objective.

4.3 Simulated Annealing

Classical simulated annealing is run on the QUBO model using the D-Wave Ocean-compatible simulated annealing tooling. It is a heuristic method. It can find exact solutions quickly on some instances, but it does not guarantee success.

4.4 Local Simulated Quantum Annealing Style Solver

The project also includes a local simulated quantum annealing style solver. It uses multiple Trotter replicas, coupling between neighboring replicas, a decreasing transverse-field schedule, and Metropolis-style bit-flip acceptance.

This method is not a physical D-Wave QPU run. It is a classical local simulation inspired by quantum annealing. Its runtime therefore reflects Python implementation overhead and the cost of simulating multiple replicas, not actual quantum hardware performance.

5 Implementation and Validation

The project is implemented as a Python package with command-line entry points for running the demonstration, benchmark, summary generation, plot generation, and benchmark validation. The benchmark configuration is stored in a JSON file, which makes the experiment reproducible.

The validation layer checks that:

- all expected benchmark columns are present,
- `exact_hit` matches the condition $E(x) = 0$,
- `success` is consistent with `exact_hit`,
- Hamming distances are within the range $[0, n]$,
- a planted-vector match cannot occur without an exact hit,
- the benchmark grid matches the configured sources, sizes, trials, and solvers.

The test suite also checks Merkle–Hellman key generation and decryption, subset-sum instance validation, QUBO energy equivalence, solver behavior, benchmark aggregation, plotting, and validation logic. At the time of writing, the full test suite contains 48 passing tests.

6 Experimental Setup

The benchmark compares two sources of subset-sum instances:

- random subset-sum instances,
- Merkle–Hellman-derived subset-sum instances.

The tested sizes are

$$n \in \{8, 12, 16\}.$$

Each source-size-solver combination is evaluated over five trials.

Table 1: Benchmark configuration used for the report.

Parameter	Value
Problem sizes	8, 12, 16 bits
Trials per setting	5
Base seed	20260504
Sources	random, Merkle–Hellman
Solvers	brute force, simulated annealing, SQA style
SA reads	100
SA sweeps	500
SQA reads	20
SQA sweeps	100
SQA Trotter slices	6
SQA beta	0.05
Random weight range	1 to 1,000,000
Benchmark rows	90

The main metrics are:

- **exact-hit rate**: fraction of trials with $E(x) = 0$,
- **planted-match rate**: fraction of trials where the recovered bit vector equals the planted vector,
- **median runtime**: median runtime in milliseconds,
- **median objective**: median value of $E(x)$,
- **mean Hamming distance**: mean distance between recovered and planted bit vectors.

The exact-hit rate and planted-match rate are reported separately because subset-sum instances may have multiple different bit vectors that produce the same target.

7 Results

7.1 Exact-Hit Rate

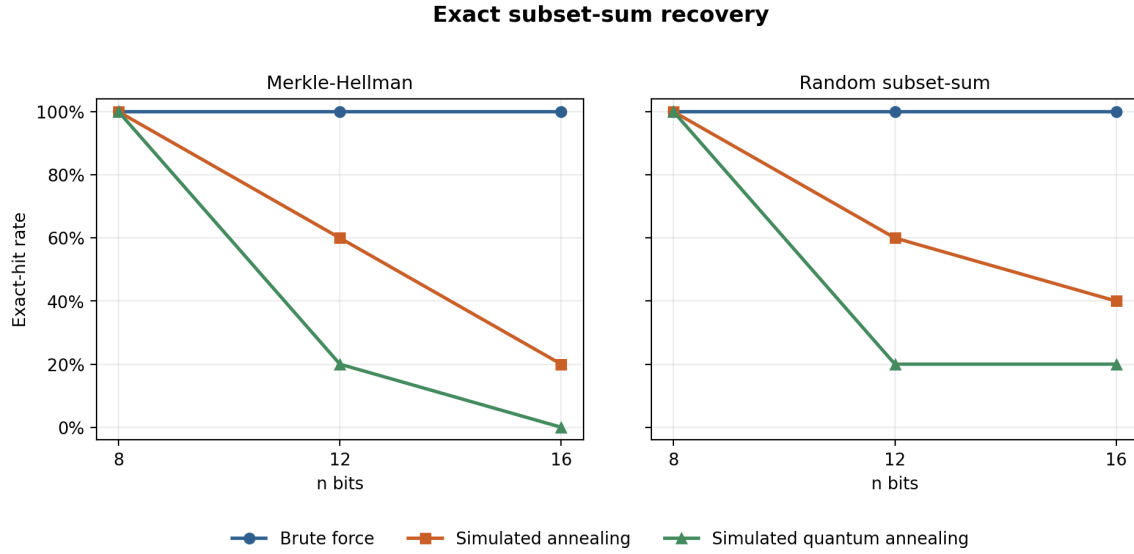


Figure 1: Exact-hit rate for random and Merkle–Hellman-derived instances. Brute force remains exact for all tested sizes, while the heuristic solvers lose reliability as n increases.

Figure 1 shows that brute force solves all tested instances exactly. This is expected because the tested instances are small enough for exhaustive search.

Simulated annealing solves all $n = 8$ instances. For Merkle–Hellman instances its exact-hit rate decreases to 0.6 at $n = 12$ and 0.2 at $n = 16$. For random instances it decreases to 0.6 at $n = 12$ and 0.4 at $n = 16$.

The local SQA-style solver also solves all $n = 8$ instances, but performs worse for larger sizes. For Merkle–Hellman instances it reaches 0.2 at $n = 12$ and 0.0 at $n = 16$. For random instances it reaches 0.2 at both $n = 12$ and $n = 16$.

7.2 Runtime

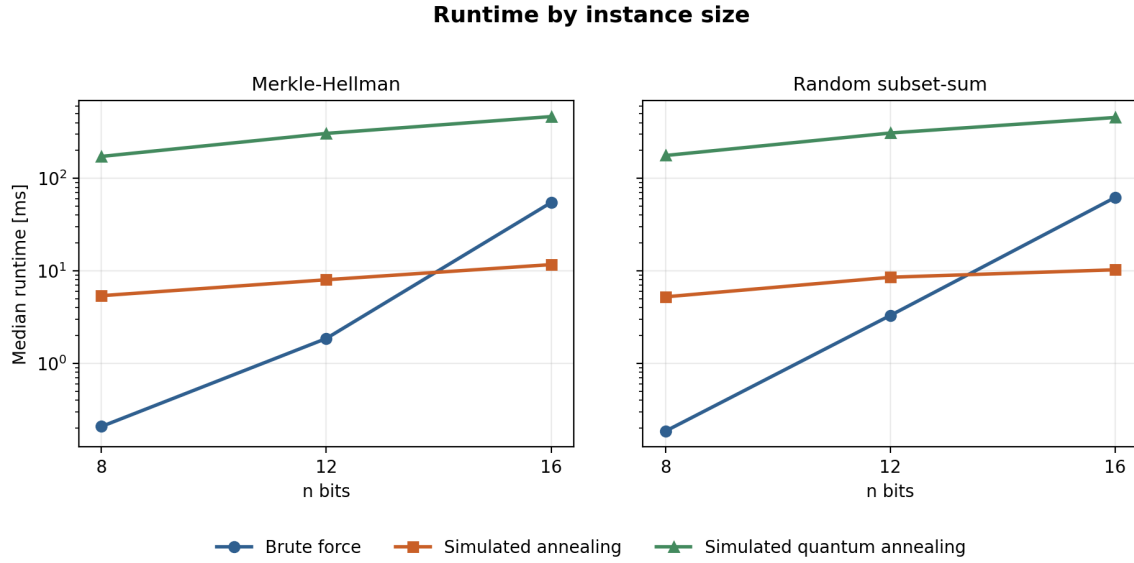


Figure 2: Median runtime on a logarithmic scale. The local SQA-style simulation is much slower than simulated annealing because it explicitly simulates multiple Trotter replicas.

Figure 2 shows the expected runtime tradeoff. Brute force grows rapidly with n , reflecting its exponential search space. At $n = 16$, the median brute force runtime is approximately 55–62 ms in this benchmark.

Simulated annealing remains fast in the tested configuration, with median runtime around 10–12 ms for $n = 16$. The local SQA-style solver is much slower, with median runtime around 455–465 ms for $n = 16$. This overhead is caused by replica simulation, not by access to quantum hardware.

7.3 Objective Gap

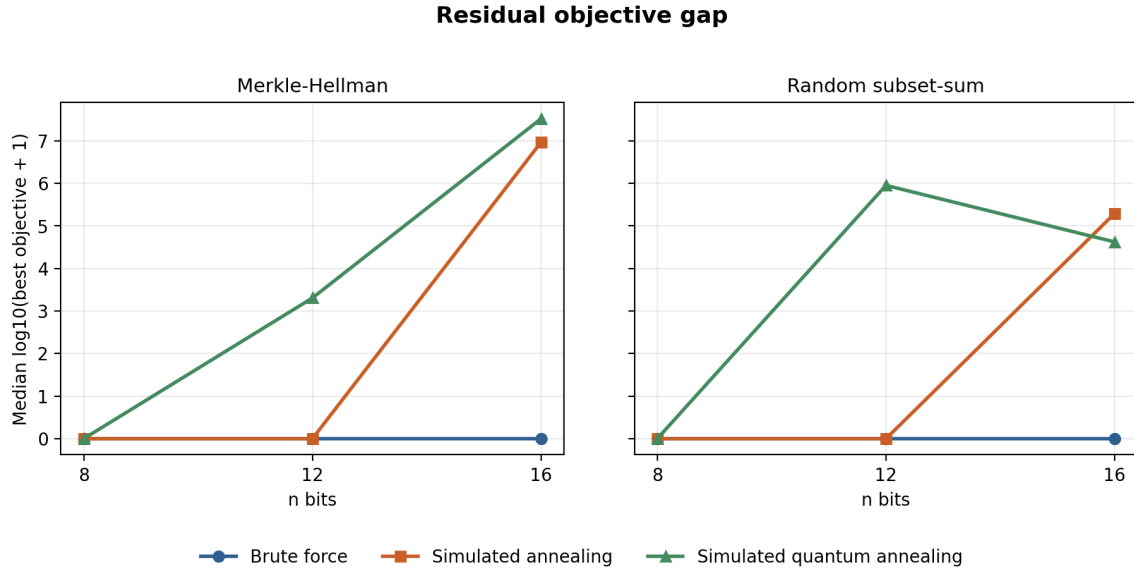


Figure 3: Median residual objective gap shown as $\log_{10}(E(x) + 1)$. A value of zero means that the median trial found an exact subset-sum solution.

Figure 3 shows how far the best recovered solutions are from the target when the solver does not hit exactly. The logarithmic transformation makes large objective gaps visible without hiding exact hits.

For $n = 16$, heuristic methods often stop with nonzero residuals. On Merkle–Hellman instances, simulated annealing has median objective value 9,302,500, while the SQA-style solver has median objective value 33,143,049.

7.4 Hamming Distance

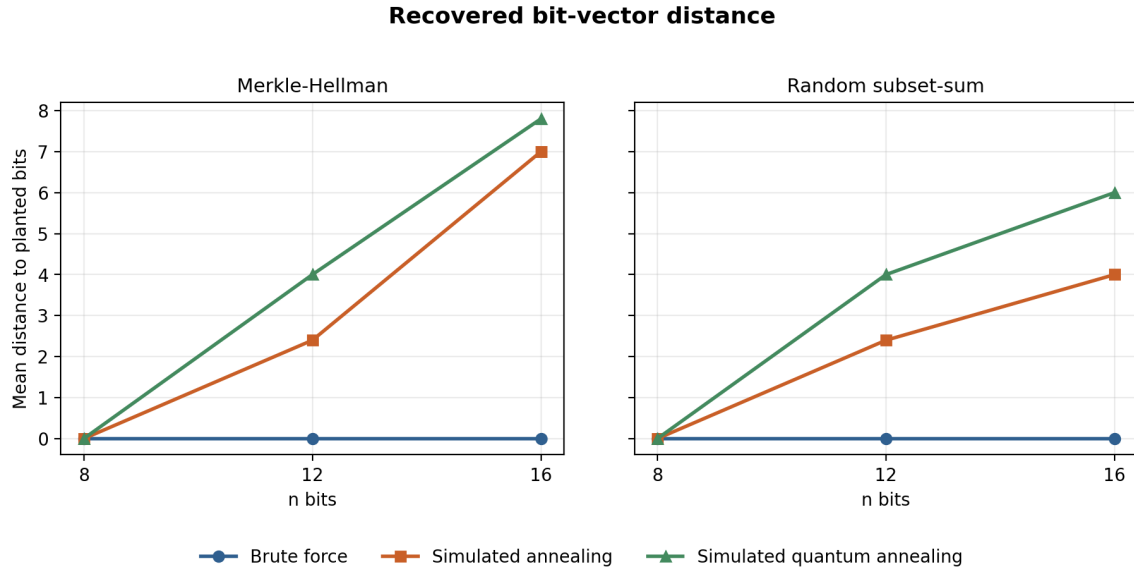


Figure 4: Mean Hamming distance between recovered and planted bit vectors. Larger values indicate that heuristic solvers recover bit strings increasingly far from the planted solution.

Figure 4 measures recovery of the planted bit vector. For $n = 8$, all methods recover the planted vector in all trials. For larger instances, the heuristic solvers increasingly return vectors several bits away from the planted solution.

On Merkle–Hellman instances with $n = 16$, simulated annealing has mean Hamming distance 7.0, while the local SQA-style solver has mean Hamming distance 7.8. This confirms that the failure cases are not merely small objective perturbations; the recovered bit vectors can differ substantially from the planted messages.

8 Numerical Summary

Table 2: Merkle–Hellman benchmark summary. Runtime is median runtime in milliseconds.

Solver	n	Hit	Match	Runtime	Objective	Hamming
Brute force	8	1.000	1.000	0.207	0.000	0.000
Brute force	12	1.000	1.000	1.855	0.000	0.000
Brute force	16	1.000	1.000	54.791	0.000	0.000
SA	8	1.000	1.000	5.383	0.000	0.000
SA	12	0.600	0.600	8.005	0.000	2.400
SA	16	0.200	0.200	11.670	9,302,500.000	7.000
SQA style	8	1.000	1.000	172.474	0.000	0.000
SQA style	12	0.200	0.200	305.768	2,025.000	4.000
SQA style	16	0.000	0.000	465.351	33,143,049.000	7.800

Table 3: Random subset-sum benchmark summary. Runtime is median runtime in milliseconds.

Solver	n	Hit	Match	Runtime	Objective	Hamming
Brute force	8	1.000	1.000	0.185	0.000	0.000
Brute force	12	1.000	1.000	3.291	0.000	0.000
Brute force	16	1.000	1.000	62.088	0.000	0.000
SA	8	1.000	1.000	5.230	0.000	0.000
SA	12	0.600	0.600	8.519	0.000	2.400
SA	16	0.400	0.400	10.261	197,136.000	4.000
SQA style	8	1.000	1.000	176.422	0.000	0.000
SQA style	12	0.200	0.200	309.028	896,809.000	4.000
SQA style	16	0.200	0.200	454.672	42,025.000	6.000

9 Discussion

The results confirm that the QUBO formulation correctly represents the subset-sum objective. Brute force consistently finds exact solutions and is therefore a useful baseline for validating the small instances.

The heuristic methods show the expected reliability tradeoff. They solve small instances, but their exact-hit rates decrease as the number of bits grows. Simulated annealing is faster than the local SQA-style simulation and performs better or comparably in the tested configuration.

The SQA-style solver should be interpreted carefully. It is a classical simulation inspired by quantum annealing, not a physical quantum annealing run. Therefore, its higher runtime does not imply that quantum annealing hardware would behave similarly. It only shows that this local Python simulation has substantial overhead.

The difference between exact-hit rate and planted-match rate is important for interpretation. Exact-hit rate measures whether the subset-sum equation was solved. Planted-match rate measures whether the original generated vector was recovered. In general subset-sum instances, these two metrics may differ because multiple bit vectors can yield the same target.

10 Limitations

The main limitations are:

- the benchmark uses only small instances,
- each benchmark point uses five trials,
- the Merkle–Hellman implementation is educational and does not represent modern cryptographic security,
- the SQA-style solver is a local simulation, not a D-Wave QPU experiment,
- heuristic performance depends on parameters such as reads, sweeps, beta, and Trotter slices.

The results should therefore be interpreted qualitatively. They demonstrate modeling correctness and solver behavior, but they are not broad statistical claims about solver performance.

11 Conclusion

This project demonstrates how a Merkle–Hellman public ciphertext can be transformed into a subset-sum instance and encoded as a QUBO problem. The mathematical model is validated through brute force, exact QUBO solving, unit tests, and benchmark validation.

The experiments show that brute force is reliable for the tested sizes but does not scale. Classical simulated annealing is fast and effective on small instances, but its success rate decreases as n increases. The local SQA-style solver does not outperform simulated annealing in the tested configuration and is significantly more expensive computationally.

Overall, the project serves as a compact educational demonstration of cryptographic problem modeling, QUBO construction, and annealing-based optimization.

References

- [1] R. C. Merkle and M. E. Hellman, “Hiding information and signatures in trapdoor knapsacks,” *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 525–530, 1978.
- [2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [3] T. Kadowaki and H. Nishimori, “Quantum annealing in the transverse Ising model,” *Physical Review E*, vol. 58, no. 5, pp. 5355–5363, 1998.
- [4] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, article 5, 2014.
- [5] D-Wave Systems, “Ocean Software Documentation,” available online: <https://docs.ocean.dwavesys.com/>.